of /photos to PhotosController#index, since :action and :id are optional parameters, denoted by parentheses.

3.2 Dynamic Segments

You can set up as many dynamic segments within a regular route as you like. Anything other than :controller or :action will be available to the action as part of params. If you set up this route:

```
get ':controller/:action/:id/:user_id'
```

An incoming path of /photos/show/1/2 will be dispatched to the show action of the PhotosController. params[:id] will be "1", and params[:user_id] will be "2".

You can't use :namespace or :module with a :controller path segment. If you need to do this then use a constraint on :controller that matches the namespace you require. e.g:

```
get ':controller(/:action(/:id))', controller: /admin\/[^\/]+/
```

By default, dynamic segments don't accept dots - this is because the dot is used as a separator for formatted routes. If you need to use a dot within a dynamic segment, add a constraint that overrides this – for example, id: /[^\/]+/ allows anything except a slash.

3.3 Static Segments

You can specify static segments when creating a route by not prepending a colon to a fragment:

```
get ':controller/:action/:id/with_user/:user_id'
```

This route would respond to paths such as /photos/show/1/with_user/2. In this case, paramswould be { controller: 'photos', action: 'show', id: '1', user_id: '2' }.

3.4 The Query String

The params will also include any parameters from the query string. For example, with this route:

```
get ':controller/:action/:id'
```

An incoming path of /photos/show/1?user_id=2 will be dispatched to the show action of the Photos controller. params will be { controller: 'photos', action: 'show', id: '1', user_id: '2' }.

3.5 Defining Defaults

You do not need to explicitly use the :controller and :action symbols within a route. You can supply them as defaults:

```
get 'photos/:id', to: 'photos#show'
```

With this route, Rails will match an incoming path of /photos/12 to the show action of PhotosController.

You can also define other defaults in a route by supplying a hash for the :defaults option. This even applies to parameters that you do not specify as dynamic segments. For example:

```
get 'photos/:id', to: 'photos#show', defaults: { format: 'jpg' }
```

Rails would match photos/12 to the show action of PhotosController, and set params[:format]to "jpg".

3.6 Naming Routes

You can specify a name for any route using the :as option:

```
get 'exit', to: 'sessions#destroy', as: :logout
```